Playwright Codegen - How to Capture Selectors

Repo: aloc999/orangehrm-ui-automation-playwright | Target: opensource-demo.orangehrmlive.com

1. What is codegen?

"Playwright codegen records every click and keystroke in the browser and automatically writes the Playwright code for it. Its only purpose is to discover the correct selectors (capturing), not to produce the final test. The raw output must always be cleaned up and moved into Page Objects."

2. How to run it

```
npm run codegen
# equivalent to:
# npx playwright codegen https://opensource-demo.orangehrmlive.com

# With the same viewport as the tests:
npx playwright codegen --viewport-size 1440,900
https://opensource-demo.orangehrmlive.com/web/index.php/auth/login
```

"Two windows open: a real browser and the codegen inspector. Every action in the browser immediately appears as a line of code in the inspector. Log in first with Admin / admin123."

3. Capturing TC001 (My Info - profile picture)

Practice: My Info > click photo > Choose File > Save

"Steps in the codegen browser: (1) click the My Info menu, (2) click the profile photo, (3) on the Change Profile Picture page click the + / Choose File button and select test-assets/profile.png, (4) click Save."

"The raw codegen output looks roughly like this:"

```
await page.getByRole('link', { name: 'My Info' }).click();
await page.locator('.orangehrm-edit-employee-image').click();
await page.locator('input[type="file"]').setInputFiles('test-assets/profile.png');
await page.getByRole('button', { name: 'Save' }).click();
```

"Key selectors discovered: .orangehrm-edit-employee-image (photo placeholder), input.oxd-file-input (hidden file input), the Save button, the Change Profile Picture heading, and the .oxd-toast--success toast containing Successfully Updated."

4. Capturing TC002 (Admin - search non-existent user)

Practice: Admin > fill Username > Search

"Steps: (1) click the Admin menu, (2) type NonExistentUserXYZ999 into the Username field, (3) click Search."

```
await page.getByRole('link', { name: 'Admin' }).click();
await page.locator('.oxd-input-group').filter({ hasText: 'Username'
}).locator('input').fill('NonExistentUserXYZ999');
await page.getByRole('button', { name: 'Search' }).click();
```

"Key selectors: the Username field inside .oxd-input-group, the Search button, and the empty state span.oxd-text--span containing No Records Found. It must be scoped to span, because the success toast uses a p tag and could false-match."

4b. Where the output appears and how to show it on camera

"You never type that block yourself. Run npm run codegen and two windows open side by side: on the left the automated Chromium browser that you click, on the right the Playwright Inspector with a code pane. Make sure the red Record dot at the top is ON. Now every click and keystroke in the left window appends one line to the right window in real time. For the video, tile the two windows next to each other and zoom the inspector to 150 percent so viewers can read along."

"Demonstrate it click by click for TC001. One, click the My Info link in the browser and point at the line that just appeared: getByRole link My Info click. Two, click the profile photo and point at the locator dot orangehrm-edit-employee-image line. Three, click Choose File, pick test-assets slash profile.png in the native dialog, and point at the setInputFiles line codegen recorded with your picked path, then explain you later replaced that machine-specific path with resolve so it works on CI too. Four, click Save and point at the getByRole button Save line. Those four born-live lines are exactly the raw block quoted above."

"Also show the three toolbar tools viewers always ask about. The Record dot pauses and resumes capturing, so pause it while typing passwords or doing irrelevant scrolling, otherwise junk lines pollute the output. The target-picker icon lets you hover any element to preview the locator Playwright would generate before you click, which is how you discover stable oxd classes versus brittle nth-child chains. The copy and clear buttons copy the whole block and wipe the pane to restart a take. Mention the language dropdown once: leave it on the default test library so the recorded syntax matches the repo."

5. From codegen output to POM (mandatory!)

"Never copy-paste raw codegen output into tests. Migrate it using this repo's pattern:

- getByRole link My Info -> becomes MyInfoPage.open() using a.oxd-main-menu-item + expectModule(/pim/i). The My Info breadcrumb reads PIM, which is invisible from codegen alone.
- locator(.orangehrm-edit-employee-image) -> becomes openProfilePictureDialog().
- setInputFiles -> becomes uploadProfilePicture(path) with resolve() for an absolute path, plus waiting for the preview .orangehrm-employee-picture img.
- Raw Save click -> becomes saveProfilePicture() with noWaitAfter: true, because the toast disappears in 5 seconds. Calling waitUntilReady first lets the toast vanish and makes the test flaky.
- Recorded toast assertion -> write it manually as .oxd-toast--success + Successfully Updated (app OS 5.9 returns Updated, not Saved). This difference is documented in the code and feature file."

6. Tips for stable selectors

"1. Prefer getByRole / getByText / stable oxd-* classes.
2. Drop fragile recorded selectors like div:nth-child(3) > input.
3. For file upload, never click the Choose File button (it opens an OS dialog Playwright cannot control); call setInputFiles on input[type=file] directly.
4. For toast / empty-state assertions, always scope the tag (span vs p) and verify with a headed run: npm run test:headed.
5. Final verification: npm run typecheck, npx bddgen, then npx playwright test --grep TC001 --workers=1."